国际化学生单车租赁管理软件源代码

源代码属性结构说明

本源码稿整理 国际化学生单车租赁管理软件V1.0 的主要自研源码，技术栈包括 Node.js、Express、Vue3、Vite、MySQL、HTML5、CSS3 和 JavaScript。源码根目录分为 backend、frontend、docs、scripts、tmp 等一级目录，其中 backend 保存服务端接口、鉴权、权限、数据库连接、SQL 和初始化脚本，frontend 保存 H5 页面、路由、接口封装、国际化资源、状态管理和页面样式，scripts 保存软著材料生成辅助脚本，docs 保存软著材料、截图和图示。本文纳入后端接口主链路、数据库结构、前端页面主链路、运行配置、初始化脚本和 README 中与部署运行直接相关的内容，不包含 node_modules、dist、第三方依赖源码和构建产物。源码稿按照“后端入口与基础设施、后端业务接口、数据库脚本、前端入口与路由、前端页面与样式、运行说明”的顺序编排，便于审查人员从系统入口逐步理解业务实现。

三级目录结构

```
backend
backend/src
backend/src/routes
backend/src/middleware
backend/src/config
backend/src/utils
backend/sql
frontend
frontend/src
frontend/src/views
frontend/src/router
frontend/src/api
frontend/src/i18n
frontend/src/stores
frontend/src/assets
scripts
README.md
```

backend/package.json

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
{
  "name": "student-bike-rental-backend",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "nodemon src/server.js",
    "start": "node src/server.js",
    "db:init": "node src/utils/initDb.js",
    "seed": "node src/utils/seedUsers.js",
    "seed:demo": "node src/utils/seedDemoData.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.7",
```

```
    "express": "^4.21.2",
    "jsonwebtoken": "^9.0.2",
    "mysql2": "^3.11.5",
    "nanoid": "^5.0.9"
  },
  "devDependencies": {
    "nodemon": "^3.1.9"
  }
}
```

backend/.env.example

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
PORT=3000
JWT_SECRET=replace_with_a_long_random_secret
DB_HOST=***
DB_PORT=3306
DB_USER=root
DB_PASSWORD=***
DB_NAME=student_bike_rental
```

backend/src/app.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import authRoutes from './routes/auth.js';
import userRoutes from './routes/users.js';
import bikeRoutes from './routes/bikes.js';
import stationRoutes from './routes/stations.js';
import orderRoutes from './routes/orders.js';
import maintenanceRoutes from './routes/maintenance.js';
import announcementRoutes from './routes/announcements.js';
import dashboardRoutes from './routes/dashboard.js';

dotenv.config();

const app = express();
app.use(cors());
app.use(express.json());

app.get('/api/health', (_req, res) => res.json({ code: 200, message: 'success', data: { status: 'ok' } }));
app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);
app.use('/api/bikes', bikeRoutes);
app.use('/api/stations', stationRoutes);
app.use('/api/orders', orderRoutes);
app.use('/api/maintenance', maintenanceRoutes);
app.use('/api/announcements', announcementRoutes);
app.use('/api/dashboard', dashboardRoutes);

app.use((req, res) => res.status(404).json({ code: 404, message: `Not found: ${req.path}` }));
app.use((err, _req, res, _next) => {
  const status = err.status || 500;
```

```
  res.status(status).json({ code: status, message: err.message || 'Internal server error' });
});

export default app;
```

backend/src/server.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import app from './app.js';

const port = Number(process.env.PORT || 3000);
app.listen(port, () => {
  console.log(`Student bike rental API running at http://localhost:${port}`);
});
```

backend/src/config/db.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import mysql from 'mysql2/promise';
import dotenv from 'dotenv';

dotenv.config();

export const pool = mysql.createPool({
  host: process.env.DB_HOST || '***',
  port: Number(process.env.DB_PORT || 3306),
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '***',
  database: process.env.DB_NAME || 'student_bike_rental',
  waitForConnections: true,
  connectionLimit: 10,
  namedPlaceholders: true
});

export async function tx(work) {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const result = await work(conn);
    await conn.commit();
    return result;
  } catch (error) {
    await conn.rollback();
    throw error;
  } finally {
    conn.release();
  }
}
```

backend/src/middleware/auth.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import jwt from 'jsonwebtoken';
import { HttpError } from '../utils/errors.js';

export function auth(required = true) {
  return (req, _res, next) => {
    const header = req.headers.authorization || '';
    const token = header.startsWith('Bearer ') ? header.slice(7) : '';
    if (!token) {
      if (!required) return next();
      return next(new HttpError(401, 'Unauthorized'));
    }
    try {
      req.user = jwt.verify(token, process.env.JWT_SECRET || 'dev_secret');
      return next();
    } catch {
      return next(new HttpError(401, 'Invalid token'));
    }
  };
}
```

backend/src/middleware/role.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { HttpError } from '../utils/errors.js';

export function allow(...roles) {
  return (req, _res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return next(new HttpError(403, 'Forbidden'));
    }
    return next();
  };
}
```

backend/src/routes/auth.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { Router } from 'express';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { pool } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { asyncHandler, ok } from '../utils/response.js';
import { HttpError } from '../utils/errors.js';

const router = Router();
const publicUserFields = 'id, username, name, email, phone, role, student_no, nationality, language, status, created_at';

router.post('/register', asyncHandler(async (req, res) => {
  const { username, password, name, email, phone, student_no, nationality, language = 'zh-CN' } = req.body;
  if (!username || !password) throw new HttpError(400, 'Username and password are required');
  const hash = await bcrypt.hash(password, 10);
  await pool.execute(
```

```
    `INSERT INTO users (username, password, name, email, phone, role, student_no, nationality, language, status)
     VALUES (?, ?, ?, ?, ?, 'student', ?, ?, ?, 'active')`,
    [username, hash, name || username, email || null, phone || null, student_no || null, nationality || null, language]
  );
  ok(res, null, 'registered');
}));

router.post('/login', asyncHandler(async (req, res) => {
  const { username, password } = req.body;
  const [rows] = await pool.execute(`SELECT * FROM users WHERE username = ? AND status = 'active'`, [username]);
  const user = rows[0];
  if (!user || !(await bcrypt.compare(password || '', user.password))) {
    throw new HttpError(401, 'Invalid username or password');
  }
  const token = jwt.sign({ id: user.id, username: user.username, role: user.role }, process.env.JWT_SECRET || 'dev_secret', {
expiresIn: '7d' });
  delete user.password;
  ok(res, { token, user });
}));

router.get('/profile', auth(), asyncHandler(async (req, res) => {
  const [rows] = await pool.execute(`SELECT ${publicUserFields} FROM users WHERE id = ?`, [req.user.id]);
  ok(res, rows[0]);
}));

router.put('/profile', auth(), asyncHandler(async (req, res) => {
  const { name, email, phone, nationality, language } = req.body;
  await pool.execute(
    'UPDATE users SET name=?, email=?, phone=?, nationality=?, language=? WHERE id=?',
    [name, email, phone, nationality, language, req.user.id]
  );
  ok(res);
}));

router.put('/password', auth(), asyncHandler(async (req, res) => {
  const { oldPassword, newPassword } = req.body;
  const [rows] = await pool.execute('SELECT password FROM users WHERE id=?', [req.user.id]);
  if (!rows[0] || !(await bcrypt.compare(oldPassword || '', rows[0].password))) throw new HttpError(400, 'Old password is
incorrect');
  await pool.execute('UPDATE users SET password=? WHERE id=?', [await bcrypt.hash(newPassword, 10), req.user.id]);
  ok(res);
}));

export default router;
```

backend/src/routes/users.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { Router } from 'express';
import bcrypt from 'bcryptjs';
import { pool } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { allow } from '../middleware/role.js';
import { asyncHandler, ok } from '../utils/response.js';

const router = Router();
router.use(auth(), allow('admin'));

router.get('/', asyncHandler(async (_req, res) => {
  const [rows] = await pool.execute('SELECT id,username,name,email,phone,role,student_no,nationality,language,status,created_at
FROM users ORDER BY id DESC');
```

```
  ok(res, rows);
}));

router.get('/:id', asyncHandler(async (req, res) => {
  const [rows] = await pool.execute('SELECT id,username,name,email,phone,role,student_no,nationality,language,status,created_at
FROM users WHERE id=?', [req.params.id]);
  ok(res, rows[0] || null);
}));

router.put('/:id', asyncHandler(async (req, res) => {
  const { name, email, phone, role, student_no, nationality, language, status, password } = req.body;
  if (password) {
    await pool.execute(
      'UPDATE users SET password=?, name=?, email=?, phone=?, role=?, student_no=?, nationality=?, language=?, status=? WHERE
id=?',
      [await bcrypt.hash(password, 10), name, email, phone, role, student_no, nationality, language, status, req.params.id]
    );
  } else {
    await pool.execute(
      'UPDATE users SET name=?, email=?, phone=?, role=?, student_no=?, nationality=?, language=?, status=? WHERE id=?',
      [name, email, phone, role, student_no, nationality, language, status, req.params.id]
    );
  }
  ok(res);
}));

router.delete('/:id', asyncHandler(async (req, res) => {
  await pool.execute(`UPDATE users SET status='disabled' WHERE id=?`, [req.params.id]);
  ok(res);
}));

export default router;
```

backend/src/routes/bikes.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { Router } from 'express';
import { pool } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { allow } from '../middleware/role.js';
import { asyncHandler, ok } from '../utils/response.js';

const router = Router();

router.get('/', auth(false), asyncHandler(async (req, res) => {
  const { status, station_id, keyword } = req.query;
  const where = [];
  const args = [];
  if (status) { where.push('b.status=?'); args.push(status); }
  if (station_id) { where.push('b.station_id=?'); args.push(station_id); }
  if (keyword) { where.push('(b.bike_no LIKE ? OR b.name LIKE ?)'); args.push(`%${keyword}%`, `%${keyword}%`); }
  const [rows] = await pool.execute(
    `SELECT b.*, s.name_zh station_name_zh, s.name_en station_name_en
     FROM bikes b LEFT JOIN stations s ON s.id=b.station_id
     ${where.length ? `WHERE ${where.join(' AND ')}` : ''} ORDER BY b.id DESC`,
    args
  );
  ok(res, rows);
}));

router.get('/:id', auth(false), asyncHandler(async (req, res) => {
```

```
  const [rows] = await pool.execute('SELECT * FROM bikes WHERE id=?', [req.params.id]);
  ok(res, rows[0] || null);
}));

router.post('/', auth(), allow('admin'), asyncHandler(async (req, res) => {
  const { bike_no, name, type, status = 'available', station_id, hourly_rate = 2, image_url, description } = req.body;
  await pool.execute(
    'INSERT INTO bikes (bike_no,name,type,status,station_id,hourly_rate,image_url,description) VALUES (?,?,?,?,?,?,?,?)',
    [bike_no, name, type, status, station_id, hourly_rate, image_url, description]
  );
  ok(res);
}));

router.put('/:id', auth(), allow('admin', 'staff'), asyncHandler(async (req, res) => {
  const { bike_no, name, type, status, station_id, hourly_rate, image_url, description } = req.body;
  await pool.execute(
    'UPDATE bikes SET bike_no=?, name=?, type=?, status=?, station_id=?, hourly_rate=?, image_url=?, description=? WHERE id=?',
    [bike_no, name, type, status, station_id, hourly_rate, image_url, description, req.params.id]
  );
  ok(res);
}));

router.delete('/:id', auth(), allow('admin'), asyncHandler(async (req, res) => {
  await pool.execute(`UPDATE bikes SET status='disabled' WHERE id=?`, [req.params.id]);
  ok(res);
}));

export default router;
```

backend/src/routes/stations.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { Router } from 'express';
import { pool } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { allow } from '../middleware/role.js';
import { asyncHandler, ok } from '../utils/response.js';

const router = Router();

router.get('/', auth(false), asyncHandler(async (_req, res) => {
  const [rows] = await pool.execute('SELECT * FROM stations ORDER BY id DESC');
  ok(res, rows);
}));

router.get('/:id', auth(false), asyncHandler(async (req, res) => {
  const [rows] = await pool.execute('SELECT * FROM stations WHERE id=?', [req.params.id]);
  ok(res, rows[0] || null);
}));

router.post('/', auth(), allow('admin'), asyncHandler(async (req, res) => {
  const { name_zh, name_en, address_zh, address_en, latitude, longitude, capacity } = req.body;
  await pool.execute(
    'INSERT INTO stations (name_zh,name_en,address_zh,address_en,latitude,longitude,capacity) VALUES (?,?,?,?,?,?,?)',
    [name_zh, name_en, address_zh, address_en, latitude, longitude, capacity]
  );
  ok(res);
}));

router.put('/:id', auth(), allow('admin'), asyncHandler(async (req, res) => {
  const { name_zh, name_en, address_zh, address_en, latitude, longitude, capacity } = req.body;
```

```
  await pool.execute(
    'UPDATE stations SET name_zh=?, name_en=?, address_zh=?, address_en=?, latitude=?, longitude=?, capacity=? WHERE id=?',
    [name_zh, name_en, address_zh, address_en, latitude, longitude, capacity, req.params.id]
  );
  ok(res);
}));

router.delete('/:id', auth(), allow('admin'), asyncHandler(async (req, res) => {
  await pool.execute('DELETE FROM stations WHERE id=?', [req.params.id]);
  ok(res);
}));

export default router;
```

backend/src/routes/orders.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { Router } from 'express';
import { nanoid } from 'nanoid';
import { pool, tx } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { allow } from '../middleware/role.js';
import { asyncHandler, ok } from '../utils/response.js';
import { HttpError } from '../utils/errors.js';

const router = Router();
router.use(auth());

router.get('/', allow('admin'), asyncHandler(async (_req, res) => {
  const [rows] = await pool.execute(
    `SELECT o.*, u.username, b.bike_no FROM rental_orders o
     LEFT JOIN users u ON u.id=o.user_id LEFT JOIN bikes b ON b.id=o.bike_id
     ORDER BY o.id DESC`
  );
  ok(res, rows);
}));

router.get('/my', asyncHandler(async (req, res) => {
  const [rows] = await pool.execute('SELECT * FROM rental_orders WHERE user_id=? ORDER BY id DESC', [req.user.id]);
  ok(res, rows);
}));

router.get('/current', asyncHandler(async (req, res) => {
  const [rows] = await pool.execute(`SELECT * FROM rental_orders WHERE user_id=? AND status='renting' ORDER BY id DESC LIMIT 1`,
[req.user.id]);
  ok(res, rows[0] || null);
}));

router.get('/:id', asyncHandler(async (req, res) => {
  const [rows] = await pool.execute('SELECT * FROM rental_orders WHERE id=?', [req.params.id]);
  const order = rows[0];
  if (!order) return ok(res, null);
  if (req.user.role !== 'admin' && order.user_id !== req.user.id) throw new HttpError(403, 'Forbidden');
  ok(res, order);
}));

router.post('/rent', allow('student', 'admin'), asyncHandler(async (req, res) => {
  const { bike_id, start_station_id } = req.body;
  const order = await tx(async (conn) => {
    const [active] = await conn.execute(`SELECT id FROM rental_orders WHERE user_id=? AND status='renting'`, [req.user.id]);
    if (active.length) throw new HttpError(400, 'You already have an active rental');
```

```
    const [bikes] = await conn.execute(`SELECT * FROM bikes WHERE id=? FOR UPDATE`, [bike_id]);
    const bike = bikes[0];
    if (!bike || bike.status !== 'available') throw new HttpError(400, 'Bike is not available');
    await conn.execute(`UPDATE bikes SET status='rented' WHERE id=?`, [bike_id]);
    const orderNo = `RO${Date.now()}${nanoid(6).toUpperCase()}`;
    await conn.execute(
      `INSERT INTO rental_orders (order_no,user_id,bike_id,start_time,hourly_rate,status,start_station_id)
       VALUES (?, ?, ?, NOW(), ?, 'renting', ?)`,
      [orderNo, req.user.id, bike_id, bike.hourly_rate, start_station_id || bike.station_id]
    );
    const [rows] = await conn.execute('SELECT * FROM rental_orders WHERE order_no=?', [orderNo]);
    return rows[0];
  });
  ok(res, order, 'rented');
}));

router.put('/:id/return', allow('student', 'admin'), asyncHandler(async (req, res) => {
  const { end_station_id } = req.body;
  const result = await tx(async (conn) => {
    const [orders] = await conn.execute(`SELECT * FROM rental_orders WHERE id=? FOR UPDATE`, [req.params.id]);
    const order = orders[0];
    if (!order || order.status !== 'renting') throw new HttpError(400, 'Order is not active');
    if (req.user.role !== 'admin' && order.user_id !== req.user.id) throw new HttpError(403, 'Forbidden');
    const minutesUsed = Math.ceil((Date.now() - new Date(order.start_time).getTime()) / 60000);
    const hours = Math.max(1, Math.ceil(minutesUsed / 60));
    const total = Number((hours * Number(order.hourly_rate)).toFixed(2));
    await conn.execute(
      `UPDATE rental_orders SET end_time=NOW(), duration_hours=?, total_amount=?, status='completed', end_station_id=? WHERE id=?
`,
      [hours, total, end_station_id || order.start_station_id, order.id]
    );
    await conn.execute(`UPDATE bikes SET status='available', station_id=? WHERE id=?`, [end_station_id || order.start_station_id,
order.bike_id]);
    const [rows] = await conn.execute('SELECT * FROM rental_orders WHERE id=?', [order.id]);
    return rows[0];
  });
  ok(res, result, 'returned');
}));

export default router;
```

backend/src/routes/maintenance.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { Router } from 'express';
import { pool, tx } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { allow } from '../middleware/role.js';
import { asyncHandler, ok } from '../utils/response.js';

const router = Router();
router.use(auth(), allow('admin', 'staff'));

router.get('/', asyncHandler(async (_req, res) => {
  const [rows] = await pool.execute(
    `SELECT m.*, b.bike_no, u.username staff_name
     FROM maintenance_records m
     LEFT JOIN bikes b ON b.id=m.bike_id
     LEFT JOIN users u ON u.id=m.staff_id
     ORDER BY m.id DESC`
  );
```

```
  ok(res, rows);
}));

router.post('/', asyncHandler(async (req, res) => {
  const { bike_id, staff_id, content } = req.body;
  await tx(async (conn) => {
    await conn.execute(
      `INSERT INTO maintenance_records (bike_id, staff_id, content, status, start_time)
       VALUES (?, ?, ?, 'processing', NOW())`,
      [bike_id, staff_id || req.user.id, content]
    );
    await conn.execute(`UPDATE bikes SET status='maintenance' WHERE id=?`, [bike_id]);
  });
  ok(res);
}));

router.put('/:id/finish', asyncHandler(async (req, res) => {
  await tx(async (conn) => {
    const [records] = await conn.execute('SELECT * FROM maintenance_records WHERE id=? FOR UPDATE', [req.params.id]);
    const record = records[0];
    await conn.execute(`UPDATE maintenance_records SET status='finished', end_time=NOW() WHERE id=?`, [req.params.id]);
    if (record) await conn.execute(`UPDATE bikes SET status='available' WHERE id=?`, [record.bike_id]);
  });
  ok(res);
}));

export default router;
```

backend/src/routes/dashboard.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { Router } from 'express';
import { pool } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { allow } from '../middleware/role.js';
import { asyncHandler, ok } from '../utils/response.js';

const router = Router();
router.use(auth(), allow('admin'));

router.get('/stats', asyncHandler(async (_req, res) => {
  const [[users]] = await pool.execute('SELECT COUNT(*) count FROM users');
  const [[bikes]] = await pool.execute('SELECT COUNT(*) count FROM bikes');
  const [[renting]] = await pool.execute(`SELECT COUNT(*) count FROM rental_orders WHERE status='renting'`);
  const [[income]] = await pool.execute(`SELECT COALESCE(SUM(total_amount),0) total FROM rental_orders WHERE
status='completed'`);
  ok(res, {
    users: users.count,
    bikes: bikes.count,
    activeOrders: renting.count,
    totalIncome: Number(income.total)
  });
}));

export default router;
```

backend/src/routes/announcements.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { Router } from 'express';
import { pool } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { allow } from '../middleware/role.js';
import { asyncHandler, ok } from '../utils/response.js';

const router = Router();

router.get('/', asyncHandler(async (_req, res) => {
  const [rows] = await pool.execute(`SELECT * FROM announcements WHERE status='published' ORDER BY id DESC`);
  ok(res, rows);
}));

router.get('/:id', asyncHandler(async (req, res) => {
  const [rows] = await pool.execute('SELECT * FROM announcements WHERE id=?', [req.params.id]);
  ok(res, rows[0] || null);
}));

router.post('/', auth(), allow('admin'), asyncHandler(async (req, res) => {
  const { title_zh, title_en, content_zh, content_en, status = 'published' } = req.body;
  await pool.execute(
    'INSERT INTO announcements (title_zh,title_en,content_zh,content_en,status,created_by) VALUES (?,?,?,?,?,?)',
    [title_zh, title_en, content_zh, content_en, status, req.user.id]
  );
  ok(res);
}));

router.put('/:id', auth(), allow('admin'), asyncHandler(async (req, res) => {
  const { title_zh, title_en, content_zh, content_en, status = 'published' } = req.body;
  await pool.execute(
    'UPDATE announcements SET title_zh=?, title_en=?, content_zh=?, content_en=?, status=? WHERE id=?',
    [title_zh, title_en, content_zh, content_en, status, req.params.id]
  );
  ok(res);
}));

router.delete('/:id', auth(), allow('admin'), asyncHandler(async (req, res) => {
  await pool.execute(`UPDATE announcements SET status='hidden' WHERE id=?`, [req.params.id]);
  ok(res);
}));

export default router;
```

backend/sql/schema.sql

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
CREATE DATABASE IF NOT EXISTS student_bike_rental DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE student_bike_rental;

CREATE TABLE IF NOT EXISTS users (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  name VARCHAR(100),
  email VARCHAR(120),
  phone VARCHAR(30),
  role ENUM('student','admin','staff') NOT NULL DEFAULT 'student',
  student_no VARCHAR(50),
```

```
  nationality VARCHAR(80),
  language VARCHAR(20) DEFAULT 'zh-CN',
  status ENUM('active','disabled') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS stations (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name_zh VARCHAR(120) NOT NULL,
  name_en VARCHAR(120) NOT NULL,
  address_zh VARCHAR(255),
  address_en VARCHAR(255),
  latitude DECIMAL(10,7),
  longitude DECIMAL(10,7),
  capacity INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS bikes (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  bike_no VARCHAR(60) NOT NULL UNIQUE,
  name VARCHAR(120),
  type VARCHAR(60),
  status ENUM('available','rented','maintenance','disabled') NOT NULL DEFAULT 'available',
  station_id BIGINT,
  hourly_rate DECIMAL(10,2) NOT NULL DEFAULT 2.00,
  image_url VARCHAR(500),
  description TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_bikes_station FOREIGN KEY (station_id) REFERENCES stations(id) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS rental_orders (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  order_no VARCHAR(80) NOT NULL UNIQUE,
  user_id BIGINT NOT NULL,
  bike_id BIGINT NOT NULL,
  start_time DATETIME NOT NULL,
  end_time DATETIME,
  duration_hours INT,
  hourly_rate DECIMAL(10,2) NOT NULL DEFAULT 2.00,
  total_amount DECIMAL(10,2) DEFAULT 0,
  status ENUM('renting','completed','cancelled') NOT NULL DEFAULT 'renting',
  start_station_id BIGINT,
  end_station_id BIGINT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_orders_user FOREIGN KEY (user_id) REFERENCES users(id),
  CONSTRAINT fk_orders_bike FOREIGN KEY (bike_id) REFERENCES bikes(id),
  INDEX idx_orders_user_status (user_id, status)
);

CREATE TABLE IF NOT EXISTS maintenance_records (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  bike_id BIGINT NOT NULL,
  staff_id BIGINT,
  content TEXT NOT NULL,
  status ENUM('processing','finished') NOT NULL DEFAULT 'processing',
  start_time DATETIME,
  end_time DATETIME,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_maintenance_bike FOREIGN KEY (bike_id) REFERENCES bikes(id),
```

```
  CONSTRAINT fk_maintenance_staff FOREIGN KEY (staff_id) REFERENCES users(id)
);

CREATE TABLE IF NOT EXISTS announcements (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  title_zh VARCHAR(200) NOT NULL,
  title_en VARCHAR(200) NOT NULL,
  content_zh TEXT NOT NULL,
  content_en TEXT NOT NULL,
  status ENUM('published','hidden') NOT NULL DEFAULT 'published',
  created_by BIGINT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_announcements_user FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL
);
```

backend/sql/seed.sql

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
USE student_bike_rental;

INSERT INTO stations (name_zh, name_en, address_zh, address_en, latitude, longitude, capacity) VALUES
('主教学楼站', 'Main Building Station', '主教学楼东门', 'East gate of Main Building', 31.2304000, 121.4737000, 40),
('国际学生公寓站', 'International Dormitory Station', '国际学生公寓入口', 'Entrance of International Dormitory', 31.2311000,
121.4750000, 30),
('图书馆站', 'Library Station', '图书馆南广场', 'South square of Library', 31.2298000, 121.4729000, 35)
ON DUPLICATE KEY UPDATE name_zh=VALUES(name_zh);

INSERT INTO bikes (bike_no, name, type, status, station_id, hourly_rate, image_url, description) VALUES
('BIKE-1001', 'Campus Bike 1001', 'standard', 'available', 1, 2.00, '', 'Standard campus rental bike'),
('BIKE-1002', 'Campus Bike 1002', 'standard', 'available', 1, 2.00, '', 'Standard campus rental bike'),
('BIKE-2001', 'City Bike 2001', 'city', 'available', 2, 3.00, '', 'Comfort city bike'),
('BIKE-3001', 'Maintenance Bike 3001', 'standard', 'maintenance', 3, 2.00, '', 'Bike waiting for maintenance')
ON DUPLICATE KEY UPDATE name=VALUES(name), status=VALUES(status);

INSERT INTO announcements (title_zh, title_en, content_zh, content_en, status, created_by) VALUES
('欢迎使用校园单车租赁系统', 'Welcome to Campus Bike Rental', '请遵守校园骑行规则，按时归还车辆。', 'Please follow campus riding
rules and return bikes on time.', 'published', NULL),
('雨天骑行提醒', 'Rainy Day Riding Notice', '雨天路滑，请降低车速并注意安全。', 'Roads may be slippery on rainy days. Please slow
down and ride safely.', 'published', NULL);

-- 初始化用户请运行：npm run seed
-- admin/***, student/***, staff/*** 会以 bcrypt hash 写入。
```

backend/src/utils/initDb.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import fs from 'node:fs/promises';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import mysql from 'mysql2/promise';
import dotenv from 'dotenv';

dotenv.config();
```

```
const __dirname = path.dirname(fileURLToPath(import.meta.url));
const root = path.resolve(__dirname, '../..');

const connection = await mysql.createConnection({
  host: process.env.DB_HOST || '***',
  port: Number(process.env.DB_PORT || 3306),
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '***',
  multipleStatements: true
});

for (const file of ['schema.sql', 'seed.sql']) {
  const sql = await fs.readFile(path.join(root, 'sql', file), 'utf8');
  await connection.query(sql);
  console.log(`Executed ${file}`);
}

await connection.end();
console.log('Database initialized.');
```

backend/src/utils/seedDemoData.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import bcrypt from 'bcryptjs';
import { pool, tx } from '../config/db.js';

const demoUsers = [
  ['admin', '***', '系统管理员', 'admin', '***', '***', null, 'China'],
  ['student', '***', '张同学', 'student', '***', '***', 'S20260001', 'China'],
  ['staff', '***', '维修人员', 'staff', '***', '***', null, 'China'],
  ['lihua', '***', '李华', 'student', '***', '***', 'S20260002', 'China'],
  ['emma', '***', 'Emma Smith', 'student', '***', '***', 'S20260003', 'United Kingdom'],
  ['akiko', '***', 'Akiko Tanaka', 'student', '***', '***', 'S20260004', 'Japan']
];

const stations = [
  ['主教学楼站', 'Main Building Station', '主教学楼东门', 'East gate of Main Building', 31.2304, 121.4737, 40],
  ['国际学生公寓站', 'International Dormitory Station', '国际学生公寓入口', 'Entrance of International Dormitory', 31.2311,
121.4750, 30],
  ['图书馆站', 'Library Station', '图书馆南广场', 'South square of Library', 31.2298, 121.4729, 35],
  ['体育馆站', 'Gymnasium Station', '体育馆北侧', 'North side of Gymnasium', 31.2287, 121.4718, 26],
  ['实验楼站', 'Laboratory Building Station', '实验楼西门', 'West gate of Laboratory Building', 31.2320, 121.4762, 22]
];

const bikes = [
  ['BIKE-1001', 'Campus Bike 1001', 'standard', 'available', 1, 2, '标准校园单车'],
  ['BIKE-1002', 'Campus Bike 1002', 'standard', 'available', 1, 2, '标准校园单车'],
  ['BIKE-1003', 'Campus Bike 1003', 'standard', 'available', 1, 2, '标准校园单车'],
  ['BIKE-2001', 'City Bike 2001', 'city', 'available', 2, 3, '舒适城市通勤车'],
  ['BIKE-2002', 'City Bike 2002', 'city', 'available', 2, 3, '舒适城市通勤车'],
  ['BIKE-3001', 'Maintenance Bike 3001', 'standard', 'maintenance', 3, 2, '待维护车辆'],
  ['BIKE-4001', 'Sport Bike 4001', 'sport', 'available', 4, 4, '轻量运动单车'],
  ['BIKE-5001', 'Campus Bike 5001', 'standard', 'disabled', 5, 2, '停用测试车辆']
];

await tx(async (conn) => {
  for (const [username, password, name, role, email, phone, studentNo, nationality] of demoUsers) {
    const hash = await bcrypt.hash(password, 10);
    await conn.execute(
      `INSERT INTO users (username,password,name,email,phone,role,student_no,nationality,language,status)
       VALUES (?,?,?,?,?,?,?,?, 'zh-CN', 'active')
```

```
       ON DUPLICATE KEY UPDATE password=VALUES(password), name=VALUES(name), email=VALUES(email), phone=VALUES(phone),
       role=VALUES(role), student_no=VALUES(student_no), nationality=VALUES(nationality), status='active'`,
      [username, hash, name, email, phone, role, studentNo, nationality]
    );
  }

  for (const station of stations) {
    await conn.execute(
      `INSERT INTO stations (name_zh,name_en,address_zh,address_en,latitude,longitude,capacity)
       SELECT ?,?,?,?,?,?,?
       WHERE NOT EXISTS (SELECT 1 FROM stations WHERE name_en=?)`,
      [...station, station[1]]
    );
  }

  for (const [bikeNo, name, type, status, stationId, rate, description] of bikes) {
    await conn.execute(
      `INSERT INTO bikes (bike_no,name,type,status,station_id,hourly_rate,image_url,description)
       VALUES (?,?,?,?,?,?,?,?)
       ON DUPLICATE KEY UPDATE name=VALUES(name), type=VALUES(type), status=VALUES(status), station_id=VALUES(station_id),
       hourly_rate=VALUES(hourly_rate), description=VALUES(description)`,
      [bikeNo, name, type, status, stationId, rate, '', description]
    );
  }

  await conn.execute(
    `INSERT INTO maintenance_records (bike_id, staff_id, content, status, start_time)
     SELECT b.id, u.id, '刹车系统检查与车铃更换', 'processing', DATE_SUB(NOW(), INTERVAL 2 HOUR)
     FROM bikes b JOIN users u ON u.username='staff'
     WHERE b.bike_no='BIKE-3001'
       AND NOT EXISTS (SELECT 1 FROM maintenance_records m WHERE m.bike_id=b.id AND m.status='processing')`
  );

  await conn.execute(
    `INSERT INTO rental_orders
(order_no,user_id,bike_id,start_time,end_time,duration_hours,hourly_rate,total_amount,status,start_station_id,end_station_id)
     SELECT 'RO-DEMO-001', u.id, b.id, DATE_SUB(NOW(), INTERVAL 5 HOUR), DATE_SUB(NOW(), INTERVAL 3 HOUR), 2, b.hourly_rate,
b.hourly_rate * 2, 'completed', 1, 2
     FROM users u JOIN bikes b ON b.bike_no='BIKE-1002'
     WHERE u.username='student'
       AND NOT EXISTS (SELECT 1 FROM rental_orders WHERE order_no='RO-DEMO-001')`
  );

  await conn.execute(
    `INSERT INTO rental_orders
(order_no,user_id,bike_id,start_time,end_time,duration_hours,hourly_rate,total_amount,status,start_station_id,end_station_id)
     SELECT 'RO-DEMO-002', u.id, b.id, DATE_SUB(NOW(), INTERVAL 1 DAY), DATE_SUB(NOW(), INTERVAL 22 HOUR), 2, b.hourly_rate,
b.hourly_rate * 2, 'completed', 2, 3
     FROM users u JOIN bikes b ON b.bike_no='BIKE-2001'
     WHERE u.username='emma'
       AND NOT EXISTS (SELECT 1 FROM rental_orders WHERE order_no='RO-DEMO-002')`
  );

  await conn.execute(
    `INSERT INTO announcements (title_zh,title_en,content_zh,content_en,status,created_by)
     SELECT '国际学生骑行安全周', 'International Student Cycling Safety Week',
            '本周将在主教学楼站开展骑行安全宣传，请同学们合理规划用车时间。',
            'A cycling safety campaign will be held at the Main Building Station this week. Please plan your rentals
accordingly.',
            'published', u.id
     FROM users u
     WHERE u.username='admin'
       AND NOT EXISTS (SELECT 1 FROM announcements WHERE title_en='International Student Cycling Safety Week')`
  );
});
```

```
console.log('Demo data seeded.');
await pool.end();
```

backend/src/utils/seedUsers.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import bcrypt from 'bcryptjs';
import { pool } from '../config/db.js';

const users = [
  ['admin', '***', 'System Admin', 'admin'],
  ['student', '***', 'Demo Student', 'student'],
  ['staff', '***', 'Maintenance Staff', 'staff']
];

for (const [username, password, name, role] of users) {
  const hash = await bcrypt.hash(password, 10);
  await pool.execute(
    `INSERT INTO users (username,password,name,role,status,language)
     VALUES (?,?,?,?, 'active', 'zh-CN')
     ON DUPLICATE KEY UPDATE password=VALUES(password), name=VALUES(name), role=VALUES(role), status='active'`,
    [username, hash, name, role]
  );
}

console.log('Seed users completed.');
await pool.end();
```

backend/src/utils/response.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
export function ok(res, data = null, message = 'success') {
  return res.json({ code: 200, message, data });
}

export function fail(res, status = 400, message = 'error') {
  return res.status(status).json({ code: status, message });
}

export function asyncHandler(fn) {
  return (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
}
```

backend/src/utils/errors.js

文件职责：该文件属于 后端服务模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
export class HttpError extends Error {
  constructor(status, message) {
    super(message);
    this.status = status;
```

```
  }
}
```

frontend/package.json

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
{
  "name": "student-bike-rental-frontend",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite --host 0.0.0.0",
    "build": "vite build",
    "preview": "vite preview --host 0.0.0.0"
  },
  "dependencies": {
    "@vitejs/plugin-vue": "^5.2.1",
    "vite": "^6.0.3",
    "vue": "^3.5.13",
    "vue-router": "^4.5.0"
  },
  "devDependencies": {}
}
```

frontend/vite.config.js

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { defineConfig } from 'vite';
import vue from '@vitejs/plugin-vue';

export default defineConfig({
  plugins: [vue()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://***',
        changeOrigin: true
      }
    }
  }
});
```

frontend/index.html

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8" />
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Student Bike Rental</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

frontend/src/main.js

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { createApp } from 'vue';
import App from './App.vue';
import router from './router/index.js';
import './assets/style.css';

createApp(App).use(router).mount('#app');
```

frontend/src/App.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <div class="app-shell">
    <header class="topbar">
      <router-link class="brand" to="/">{{ t('app') }}</router-link>
      <nav class="toolbar">
        <router-link to="/">{{ t('home') }}</router-link>
        <router-link v-if="auth.user" to="/bikes">{{ t('bikes') }}</router-link>
        <router-link v-if="auth.user" to="/orders/current">{{ t('currentOrder') }}</router-link>
        <router-link v-if="auth.user" to="/orders/history">{{ t('history') }}</router-link>
        <router-link to="/announcements">{{ t('announcements') }}</router-link>
        <router-link v-if="auth.user" to="/profile">{{ t('profile') }}</router-link>
        <router-link v-if="auth.user?.role === 'admin'" to="/admin">{{ t('dashboard') }}</router-link>
        <router-link v-if="!auth.user" to="/login">{{ t('login') }}</router-link>
        <button v-if="auth.user" @click="logout">{{ t('logout') }}</button>
        <select :value="state.lang" @change="setLang($event.target.value)">
          <option value="zh-CN">中文</option>
          <option value="en-US">English</option>
        </select>
      </nav>
    </header>
    <main class="content">
      <router-view />
    </main>
  </div>
</template>

<script setup>
import { useRouter } from 'vue-router';
import { authStore as auth } from './stores/auth.js';
import { state, t, setLang } from './i18n/index.js';

const router = useRouter();
function logout() {
```

```
  auth.logout();
  router.push('/login');
}
</script>
```

frontend/src/router/index.js

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { createRouter, createWebHistory } from 'vue-router';
import { authStore } from '../stores/auth.js';
import Login from '../views/Login.vue';
import Register from '../views/Register.vue';
import Home from '../views/Home.vue';
import Bikes from '../views/Bikes.vue';
import BikeDetail from '../views/BikeDetail.vue';
import RentConfirm from '../views/RentConfirm.vue';
import CurrentOrder from '../views/CurrentOrder.vue';
import Orders from '../views/Orders.vue';
import Profile from '../views/Profile.vue';
import Announcements from '../views/Announcements.vue';
import AnnouncementDetail from '../views/AnnouncementDetail.vue';
import AdminList from '../views/AdminList.vue';
import Dashboard from '../views/Dashboard.vue';

const routes = [
  { path: '/', component: Home },
  { path: '/login', component: Login },
  { path: '/register', component: Register },
  { path: '/bikes', component: Bikes, meta: { auth: true } },
  { path: '/bikes/:id', component: BikeDetail, meta: { auth: true } },
  { path: '/rent/:id', component: RentConfirm, meta: { auth: true } },
  { path: '/orders/current', component: CurrentOrder, meta: { auth: true } },
  { path: '/orders/history', component: Orders, meta: { auth: true } },
  { path: '/profile', component: Profile, meta: { auth: true } },
  { path: '/announcements', component: Announcements },
  { path: '/announcements/:id', component: AnnouncementDetail },
  { path: '/admin', component: Dashboard, meta: { auth: true, admin: true } },
  { path: '/admin/users', component: AdminList, props: { type: 'users' }, meta: { auth: true, admin: true } },
  { path: '/admin/bikes', component: AdminList, props: { type: 'bikes' }, meta: { auth: true, admin: true } },
  { path: '/admin/orders', component: AdminList, props: { type: 'orders' }, meta: { auth: true, admin: true } },
  { path: '/admin/stations', component: AdminList, props: { type: 'stations' }, meta: { auth: true, admin: true } },
  { path: '/admin/maintenance', component: AdminList, props: { type: 'maintenance' }, meta: { auth: true } },
  { path: '/admin/announcements', component: AdminList, props: { type: 'announcements' }, meta: { auth: true, admin: true } }
];

const router = createRouter({ history: createWebHistory(), routes });

router.beforeEach((to) => {
  if (to.meta.auth && !authStore.isAuthed) return '/login';
  if (to.meta.admin && authStore.user?.role !== 'admin') return '/';
  return true;
});

export default router;
```

frontend/src/api/client.js

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
const API_BASE = import.meta.env.VITE_API_BASE || 'http://***/api';

export function token() {
  return localStorage.getItem('token') || '';
}

export async function request(path, options = {}) {
  const headers = { 'Content-Type': 'application/json', ...(options.headers || {}) };
  if (token()) headers.Authorization = `Bearer ${token()}`;
  const response = await fetch(`${API_BASE}${path}`, {
    ...options,
    headers,
    body: options.body ? JSON.stringify(options.body) : undefined
  });
  const payload = await response.json().catch(() => ({ message: 'Network error' }));
  if (!response.ok || payload.code >= 400) throw new Error(payload.message || 'Request failed');
  return payload.data;
}
```

frontend/src/i18n/index.js

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { reactive } from 'vue';

export const state = reactive({
  lang: localStorage.getItem('lang') || 'zh-CN'
});

const messages = {
  'zh-CN': {
    app: '国际化学生单车租赁',
    login: '登录',
    register: '注册',
    logout: '退出',
    home: '首页',
    bikes: '单车',
    currentOrder: '当前订单',
    history: '历史订单',
    profile: '个人中心',
    announcements: '公告',
    dashboard: '后台',
    users: '用户',
    orders: '订单',
    stations: '站点',
    maintenance: '维护',
    rent: '租赁',
    returnBike: '归还',
    save: '保存',
    search: '搜索',
    username: '用户名',
    password: '密码',
    name: '姓名',
    email: '邮箱',
    phone: '电话',
    status: '状态',
    available: '可租赁',
    rented: '已租赁',
    disabled: '停用',
```

```
    maintenanceStatus: '维护中',
    language: '语言',
    detail: '详情',
    all: '全部',
    create: '新增',
    action: '操作',
    finish: '完成',
    delete: '删除',
    noActiveOrder: '暂无当前订单',
    confirmReturn: '确认归还？',
    completedTotal: '已完成，费用：',
    bike: '车辆',
    id: '编号',
    bike_no: '车辆编号',
    order_no: '订单编号',
    role: '角色',
    type: '类型',
    hourly_rate: '小时费率',
    total_amount: '总费用',
    capacity: '容量',
    name_zh: '中文名称',
    name_en: '英文名称',
    staff_name: '维护人员',
    content: '内容',
    title_zh: '中文标题',
    title_en: '英文标题'
  },
  'en-US': {
    app: 'International Student Bike Rental',
    login: 'Login',
    register: 'Register',
    logout: 'Logout',
    home: 'Home',
    bikes: 'Bikes',
    currentOrder: 'Current',
    history: 'History',
    profile: 'Profile',
    announcements: 'Announcements',
    dashboard: 'Admin',
    users: 'Users',
    orders: 'Orders',
    stations: 'Stations',
    maintenance: 'Maintenance',
    rent: 'Rent',
    returnBike: 'Return',
    save: 'Save',
    search: 'Search',
    username: 'Username',
    password: 'Password',
    name: 'Name',
    email: 'Email',
    phone: 'Phone',
    status: 'Status',
    available: 'Available',
    rented: 'Rented',
    disabled: 'Disabled',
    maintenanceStatus: 'Maintenance',
    language: 'Language',
    detail: 'Detail',
    all: 'All',
    create: 'Create',
    action: 'Action',
    finish: 'Finish',
    delete: 'Delete',
    noActiveOrder: 'No active order',
    confirmReturn: 'Confirm return?',
```

```
    completedTotal: 'Completed. Total: ',
    bike: 'Bike',
    id: 'ID',
    bike_no: 'Bike No.',
    order_no: 'Order No.',
    role: 'Role',
    type: 'Type',
    hourly_rate: 'Hourly Rate',
    total_amount: 'Total Amount',
    capacity: 'Capacity',
    name_zh: 'Chinese Name',
    name_en: 'English Name',
    staff_name: 'Staff',
    content: 'Content',
    title_zh: 'Chinese Title',
    title_en: 'English Title'
  }
};

export function t(key) {
  return messages[state.lang]?.[key] || key;
}

export function setLang(lang) {
  state.lang = lang;
  localStorage.setItem('lang', lang);
}
```

frontend/src/stores/auth.js

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
import { reactive } from 'vue';
import { request } from '../api/client.js';

export const authStore = reactive({
  user: JSON.parse(localStorage.getItem('user') || 'null'),
  get isAuthed() {
    return Boolean(localStorage.getItem('token'));
  },
  async login(form) {
    const data = await request('/auth/login', { method: 'POST', body: form });
    localStorage.setItem('token', data.token);
    localStorage.setItem('user', JSON.stringify(data.user));
    this.user = data.user;
  },
  logout() {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    this.user = null;
  }
});
```

frontend/src/views/Login.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <section>
    <h1>{{ t('login') }}</h1>
    <form class="form" @submit.prevent="submit">
      <input v-model="form.username" :placeholder="t('username')" required />
      <input v-model="form.password" :placeholder="t('password')" type="password" required />
      <button class="btn">{{ t('login') }}</button>
      <router-link to="/register">{{ t('register') }}</router-link>
      <p class="muted">{{ message }}</p>
    </form>
  </section>
</template>
<script setup>
import { reactive, ref } from 'vue';
import { useRouter } from 'vue-router';
import { authStore } from '../stores/auth.js';
import { t } from '../i18n/index.js';
const router = useRouter();
const message = ref('');
const form = reactive({ username: 'student', password: '***' });
async function submit() {
  try {
    await authStore.login(form);
    router.push('/');
  } catch (e) {
    message.value = e.message;
  }
}
</script>
```

frontend/src/views/Register.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <section>
    <h1>{{ t('register') }}</h1>
    <form class="form" @submit.prevent="submit">
      <input v-model="form.username" :placeholder="t('username')" required />
      <input v-model="form.password" :placeholder="t('password')" type="password" required />
      <input v-model="form.name" :placeholder="t('name')" />
      <input v-model="form.email" :placeholder="t('email')" />
      <input v-model="form.phone" :placeholder="t('phone')" />
      <input v-model="form.student_no" placeholder="Student No." />
      <input v-model="form.nationality" placeholder="Nationality" />
      <button class="btn">{{ t('register') }}</button>
      <p class="muted">{{ message }}</p>
    </form>
  </section>
</template>
<script setup>
import { reactive, ref } from 'vue';
import { request } from '../api/client.js';
import { t, state } from '../i18n/index.js';
const message = ref('');
const form = reactive({ username: '', password: '', name: '', email: '', phone: '', student_no: '', nationality: '', language:
state.lang });
async function submit() {
  try {
    await request('/auth/register', { method: 'POST', body: form });
    message.value = 'registered';
```

```
  } catch (e) {
    message.value = e.message;
  }
}
</script>
```

frontend/src/views/Home.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <section>
    <h1>{{ t('app') }}</h1>
    <div class="grid">
      <router-link class="card" to="/bikes"><h3>{{ t('bikes') }}</h3><p class="muted">Campus bikes and station availability</p>
</router-link>
      <router-link class="card" to="/orders/current"><h3>{{ t('currentOrder') }}</h3><p class="muted">Active rental and return
action</p></router-link>
      <router-link class="card" to="/announcements"><h3>{{ t('announcements') }}</h3><p class="muted">Campus rental notices</p>
</router-link>
      <router-link v-if="auth.user?.role === 'admin'" class="card" to="/admin"><h3>{{ t('dashboard') }}</h3><p
class="muted">Management console</p></router-link>
    </div>
  </section>
</template>
<script setup>
import { t } from '../i18n/index.js';
import { authStore as auth } from '../stores/auth.js';
</script>
```

frontend/src/views/Bikes.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <section>
    <div class="row">
      <h1>{{ t('bikes') }}</h1>
      <input v-model="keyword" :placeholder="t('search')" @keyup.enter="load" />
      <select v-model="status" @change="load">
        <option value="">{{ t('all') }}</option>
        <option value="available">{{ t('available') }}</option>
        <option value="rented">{{ t('rented') }}</option>
        <option value="maintenance">{{ t('maintenanceStatus') }}</option>
      </select>
      <button class="btn secondary" @click="load">{{ t('search') }}</button>
    </div>
    <div class="grid">
      <article v-for="bike in bikes" :key="bike.id" class="card">
        <h3>{{ bike.name || bike.bike_no }}</h3>
        <p>{{ bike.bike_no }} · {{ bike.type }}</p>
        <p><span class="status" :class="bike.status">{{ t(bike.status) }}</span></p>
        <p class="muted">{{ bike.station_name_zh || bike.station_name_en }}</p>
        <p>¥{{ bike.hourly_rate }}/h</p>
        <router-link class="btn" :to="`/bikes/${bike.id}`">{{ t('detail') }}</router-link>
      </article>
    </div>
```

```
  </section>
</template>
<script setup>
import { onMounted, ref } from 'vue';
import { request } from '../api/client.js';
import { t } from '../i18n/index.js';
const bikes = ref([]);
const keyword = ref('');
const status = ref('');
async function load() {
  const params = new URLSearchParams();
  if (keyword.value) params.set('keyword', keyword.value);
  if (status.value) params.set('status', status.value);
  bikes.value = await request(`/bikes?${params}`);
}
onMounted(load);
</script>
```

frontend/src/views/BikeDetail.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <section v-if="bike">
    <h1>{{ bike.name || bike.bike_no }}</h1>
    <div class="card">
      <p>{{ bike.bike_no }} · {{ bike.type }}</p>
      <p><span class="status" :class="bike.status">{{ t(bike.status) }}</span></p>
      <p>￥{{ bike.hourly_rate }}/h</p>
      <p class="muted">{{ bike.description }}</p>
      <router-link v-if="bike.status === 'available'" class="btn" :to="`/rent/${bike.id}`">{{ t('rent') }}</router-link>
    </div>
  </section>
</template>
<script setup>
import { onMounted, ref } from 'vue';
import { useRoute } from 'vue-router';
import { request } from '../api/client.js';
import { t } from '../i18n/index.js';
const route = useRoute();
const bike = ref(null);
onMounted(async () => { bike.value = await request(`/bikes/${route.params.id}`); });
</script>
```

frontend/src/views/RentConfirm.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <section>
    <h1>{{ t('rent') }}</h1>
    <div class="card" v-if="bike">
      <h3>{{ bike.name || bike.bike_no }}</h3>
      <p>￥{{ bike.hourly_rate }}/h</p>
      <button class="btn" @click="rent">{{ t('rent') }}</button>
      <p class="muted">{{ message }}</p>
    </div>
```

```
  </section>
</template>
<script setup>
import { onMounted, ref } from 'vue';
import { useRoute, useRouter } from 'vue-router';
import { request } from '../api/client.js';
import { t } from '../i18n/index.js';
const route = useRoute();
const router = useRouter();
const bike = ref(null);
const message = ref('');
onMounted(async () => { bike.value = await request(`/bikes/${route.params.id}`); });
async function rent() {
  if (!confirm('Confirm rental?')) return;
  try {
    await request('/orders/rent', { method: 'POST', body: { bike_id: Number(route.params.id), start_station_id:
bike.value.station_id } });
    router.push('/orders/current');
  } catch (e) {
    message.value = e.message;
  }
}
</script>
```

frontend/src/views/CurrentOrder.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <section>
    <h1>{{ t('currentOrder') }}</h1>
    <div class="card" v-if="order">
      <h3>{{ order.order_no }}</h3>
      <p>{{ t('bike') }} #{{ order.bike_id }}</p>
      <p>{{ order.start_time }}</p>
      <p><span class="status" :class="order.status">{{ order.status }}</span></p>
      <button class="btn danger" @click="returnBike">{{ t('returnBike') }}</button>
    </div>
    <p v-else class="muted">{{ t('noActiveOrder') }}</p>
    <p class="muted">{{ message }}</p>
  </section>
</template>
<script setup>
import { onMounted, ref } from 'vue';
import { request } from '../api/client.js';
import { t } from '../i18n/index.js';
const order = ref(null);
const message = ref('');
async function load() { order.value = await request('/orders/current'); }
async function returnBike() {
  if (!confirm(t('confirmReturn'))) return;
  try {
    const returned = await request(`/orders/${order.value.id}/return`, { method: 'PUT', body: { end_station_id:
order.value.start_station_id } });
    order.value = null;
    message.value = `${t('completedTotal')}￥${returned.total_amount}`;
  } catch (e) { message.value = e.message; }
}
onMounted(load);
</script>
```

frontend/src/views/Orders.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <section>
    <h1>{{ t('history') }}</h1>
    <div class="card" style="overflow:auto">
      <table>
        <thead><tr><th>No.</th><th>Bike</th><th>Status</th><th>Hours</th><th>Total</th></tr></thead>
        <tbody><tr v-for="o in orders" :key="o.id"><td>{{ o.order_no }}</td><td>{{ o.bike_id }}</td><td><span class="status"
:class="o.status">{{ o.status }}</span></td><td>{{ o.duration_hours || '-' }}</td><td>￥{{ o.total_amount }}</td></tr></tbody>
      </table>
    </div>
  </section>
</template>
<script setup>
import { onMounted, ref } from 'vue';
import { request } from '../api/client.js';
import { t } from '../i18n/index.js';
const orders = ref([]);
onMounted(async () => { orders.value = await request('/orders/my'); });
</script>
```

frontend/src/views/Profile.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <section>
    <h1>{{ t('profile') }}</h1>
    <form class="form" @submit.prevent="save">
      <input v-model="form.name" :placeholder="t('name')" />
      <input v-model="form.email" :placeholder="t('email')" />
      <input v-model="form.phone" :placeholder="t('phone')" />
      <input v-model="form.nationality" placeholder="Nationality" />
      <select v-model="form.language"><option value="zh-CN">中文</option><option value="en-US">English</option></select>
      <button class="btn">{{ t('save') }}</button>
      <p class="muted">{{ message }}</p>
    </form>
  </section>
</template>
<script setup>
import { onMounted, reactive, ref } from 'vue';
import { request } from '../api/client.js';
import { t } from '../i18n/index.js';
const message = ref('');
const form = reactive({ name: '', email: '', phone: '', nationality: '', language: 'zh-CN' });
onMounted(async () => Object.assign(form, await request('/auth/profile')));
async function save() {
  await request('/auth/profile', { method: 'PUT', body: form });
  message.value = 'saved';
}
</script>
```

frontend/src/views/Announcements.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <section>
    <h1>{{ t('announcements') }}</h1>
    <div class="grid">
      <router-link v-for="a in list" :key="a.id" class="card" :to="`/announcements/${a.id}`">
        <h3>{{ state.lang === 'zh-CN' ? a.title_zh : a.title_en }}</h3>
        <p class="muted">{{ a.created_at }}</p>
      </router-link>
    </div>
  </section>
</template>
<script setup>
import { onMounted, ref } from 'vue';
import { request } from '../api/client.js';
import { state, t } from '../i18n/index.js';
const list = ref([]);
onMounted(async () => { list.value = await request('/announcements'); });
</script>
```

frontend/src/views/AnnouncementDetail.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <section v-if="item">
    <h1>{{ state.lang === 'zh-CN' ? item.title_zh : item.title_en }}</h1>
    <article class="card">{{ state.lang === 'zh-CN' ? item.content_zh : item.content_en }}</article>
  </section>
</template>
<script setup>
import { onMounted, ref } from 'vue';
import { useRoute } from 'vue-router';
import { request } from '../api/client.js';
import { state } from '../i18n/index.js';
const route = useRoute();
const item = ref(null);
onMounted(async () => { item.value = await request(`/announcements/${route.params.id}`); });
</script>
```

frontend/src/views/AdminList.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <section>
    <div class="row">
      <h1>{{ title }}</h1>
      <button v-if="canCreate" class="btn" @click="createSample">{{ t('create') }}</button>
    </div>
    <p class="muted">{{ message }}</p>
    <div class="card" style="overflow:auto">
      <table>
        <thead>
          <tr><th v-for="h in headers" :key="h">{{ t(h) }}</th><th>{{ t('action') }}</th></tr>
        </thead>
```

```
        <tbody>
          <tr v-for="row in rows" :key="row.id">
            <td v-for="h in headers" :key="h">{{ format(row[h]) }}</td>
            <td>
              <button v-if="type === 'maintenance' && row.status === 'processing'" class="btn" @click="finish(row)">{{
t('finish') }}</button>
              <button v-if="deletable" class="btn danger" @click="remove(row)">{{ t('delete') }}</button>
            </td>
          </tr>
        </tbody>
      </table>
    </div>
  </section>
</template>
<script setup>
import { computed, onMounted, ref, watch } from 'vue';
import { request } from '../api/client.js';
import { t } from '../i18n/index.js';

const props = defineProps({ type: { type: String, required: true } });
const rows = ref([]);
const message = ref('');
const title = computed(() => t(props.type));
const endpoints = {
  users: '/users',
  bikes: '/bikes',
  orders: '/orders',
  stations: '/stations',
  maintenance: '/maintenance',
  announcements: '/announcements'
};
const headerMap = {
  users: ['id', 'username', 'name', 'role', 'status'],
  bikes: ['id', 'bike_no', 'name', 'type', 'status', 'hourly_rate'],
  orders: ['id', 'order_no', 'username', 'bike_no', 'status', 'total_amount'],
  stations: ['id', 'name_zh', 'name_en', 'capacity'],
  maintenance: ['id', 'bike_no', 'staff_name', 'status', 'content'],
  announcements: ['id', 'title_zh', 'title_en', 'status']
};
const headers = computed(() => headerMap[props.type] || ['id']);
const deletable = computed(() => ['users', 'bikes', 'stations', 'announcements'].includes(props.type));
const canCreate = computed(() => ['bikes', 'stations', 'maintenance', 'announcements'].includes(props.type));

async function load() {
  rows.value = await request(endpoints[props.type]);
}
function format(value) {
  if (value == null) return '';
  if (typeof value === 'string' && value.length > 60) return `${value.slice(0, 60)}...`;
  return value;
}
async function remove(row) {
  if (!confirm('Delete this item?')) return;
  await request(`${endpoints[props.type]}/${row.id}`, { method: 'DELETE' });
  await load();
}
async function finish(row) {
  await request(`/maintenance/${row.id}/finish`, { method: 'PUT' });
  await load();
}
async function createSample() {
  const samples = {
    bikes: { bike_no: `BIKE-${Date.now()}`, name: 'New Bike', type: 'standard', status: 'available', station_id: 1, hourly_rate:
2, image_url: '', description: 'Created from admin' },
    stations: { name_zh: '新站点', name_en: 'New Station', address_zh: '校园内', address_en: 'Campus', latitude: 31.23,
longitude: 121.47, capacity: 20 },
```

```
    maintenance: { bike_id: 1, content: 'Routine inspection' },
    announcements: { title_zh: '新公告', title_en: 'New Notice', content_zh: '公告内容', content_en: 'Notice content', status:
'published' }
  };
  try {
    await request(endpoints[props.type], { method: 'POST', body: samples[props.type] });
    await load();
  } catch (e) {
    message.value = e.message;
  }
}
onMounted(load);
watch(() => props.type, load);
</script>
```

frontend/src/views/Dashboard.vue

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
<template>
  <section>
    <h1>{{ t('dashboard') }}</h1>
    <div class="grid">
      <div class="card"><h3>{{ stats.users }}</h3><p>{{ t('users') }}</p></div>
      <div class="card"><h3>{{ stats.bikes }}</h3><p>{{ t('bikes') }}</p></div>
      <div class="card"><h3>{{ stats.activeOrders }}</h3><p>{{ t('currentOrder') }}</p></div>
      <div class="card"><h3>￥{{ stats.totalIncome }}</h3><p>Total income</p></div>
    </div>
    <div class="grid" style="margin-top:12px">
      <router-link class="card" to="/admin/users">{{ t('users') }}</router-link>
      <router-link class="card" to="/admin/bikes">{{ t('bikes') }}</router-link>
      <router-link class="card" to="/admin/orders">{{ t('orders') }}</router-link>
      <router-link class="card" to="/admin/stations">{{ t('stations') }}</router-link>
      <router-link class="card" to="/admin/maintenance">{{ t('maintenance') }}</router-link>
      <router-link class="card" to="/admin/announcements">{{ t('announcements') }}</router-link>
    </div>
  </section>
</template>
<script setup>
import { onMounted, reactive } from 'vue';
import { request } from '../api/client.js';
import { t } from '../i18n/index.js';
const stats = reactive({ users: 0, bikes: 0, activeOrders: 0, totalIncome: 0 });
onMounted(async () => Object.assign(stats, await request('/dashboard/stats')));
</script>
```

frontend/src/assets/style.css

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, "Microsoft YaHei", sans-serif; background: #f6f7f9; color: #172033; }
a { color: inherit; text-decoration: none; }
button, input, select, textarea { font: inherit; }
.app-shell { max-width: 980px; min-height: 100vh; margin: 0 auto; background: #fff; }
.topbar { position: sticky; top: 0; z-index: 5; display: flex; gap: 10px; align-items: center; justify-content: space-between;
padding: 12px 14px; background: #10233f; color: #fff; }
```

```
.brand { font-weight: 700; }
.toolbar { display: flex; gap: 8px; align-items: center; flex-wrap: wrap; }
.toolbar a, .toolbar button, .toolbar select { border: 0; border-radius: 6px; padding: 8px 10px; background:
rgba(255,255,255,.12); color: #fff; }
.toolbar select option { color: #172033; }
.content { padding: 16px; }
.grid { display: grid; gap: 12px; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); }
.card { border: 1px solid #e4e8ef; border-radius: 8px; padding: 14px; background: #fff; box-shadow: 0 1px 2px rgba(0,0,0,.04); }
.row { display: flex; gap: 10px; align-items: center; flex-wrap: wrap; }
.form { display: grid; gap: 12px; max-width: 480px; }
.form input, .form select, .form textarea { width: 100%; border: 1px solid #ccd4df; border-radius: 6px; padding: 10px; }
.btn { border: 0; border-radius: 6px; padding: 10px 12px; background: #0d6efd; color: #fff; cursor: pointer; }
.btn.secondary { background: #5c667a; }
.btn.danger { background: #c0392b; }
.muted { color: #687386; }
.status { display: inline-block; border-radius: 999px; padding: 3px 8px; font-size: 12px; background: #e9edf3; }
.status.available, .status.completed, .status.published { background: #dff5e8; color: #126b35; }
.status.rented, .status.renting, .status.processing { background: #fff3cd; color: #7b5800; }
.status.maintenance, .status.disabled, .status.hidden { background: #fde2e2; color: #9b1c1c; }
table { width: 100%; border-collapse: collapse; font-size: 14px; }
th, td { border-bottom: 1px solid #e7ebf0; padding: 10px; text-align: left; vertical-align: top; }
@media (max-width: 680px) {
  .topbar { align-items: flex-start; flex-direction: column; }
  .toolbar a, .toolbar button, .toolbar select { padding: 7px 8px; font-size: 13px; }
  th, td { padding: 8px 6px; font-size: 12px; }
}
```

README.md

文件职责：该文件属于 前端 H5 模块，用于支撑系统认证、业务接口、页面交互或数据结构。

```
# 国际化学生单车租赁软件

这是一个 Node.js + Express + MySQL 后端、Vue3 + Vite H5 前端的校园学生单车租赁系统。系统支持学生租车还车、订单查询、公告浏览、中英文切换，以及管理员后台管理用户、单车、订单、站点、维护和公告。

## 目录结构

```text
backend/   Node.js REST API
frontend/  Vue3 H5 前端
tmp/       本次生成需求临时稿
```

数据库配置

后端默认读取 backend/.env：

```
DB_HOST=***
DB_PORT=3306
DB_USER=root
DB_PASSWORD=***
DB_NAME=student_bike_rental
```

初始化数据库
```

```
mysql -h *** -P 3306 -u root -p < backend/sql/schema.sql
mysql -h *** -P 3306 -u root -p student_bike_rental < backend/sql/seed.sql
```

初始化测试账号需要用 bcrypt 写入，请在安装后端依赖后运行：

```
cd backend
npm install
npm run seed
```

测试账号：

角色	用户名	密码
管理员	admin	***
学生	student	***
运维	staff	***

启动后端

```
cd backend
npm run dev
```

默认地址：http://***。

启动前端

```
cd frontend
npm install
npm run dev
```

默认地址：http://***。

主要接口

- POST /api/auth/register
- POST /api/auth/login
- GET /api/auth/profile
- GET /api/bikes
- POST /api/orders/rent
- PUT /api/orders/:id/return
- GET /api/dashboard/stats

接口统一返回：

```
{
  "code": 200,
  "message": "success",
```

```
  "data": {}
}
```

已实现页面

- 登录页、注册页、首页
- 单车列表、单车详情、租赁确认
- 当前订单、历史订单、个人中心
- 公告列表、公告详情
- 管理后台、用户管理、单车管理、订单管理、站点管理、维护管理、公告管理

说明

- 租车和还车接口使用数据库事务。
- 密码使用 bcryptjs 加密。
- 登录使用 JWT。
- 前端语言选择保存在 `localStorage.lang`。
- 管理后台中的新增按钮使用演示数据快速创建记录，后续可以扩展为完整弹窗表单。

```
```